

Métodos de Ensamble: *Bagging* y *Random Forest*

Tutorial 12 – Promediar árboles para reducir varianza

Luciana Azul Ramirez

E337 – Big Data para Economistas
Universidad de San Andrés

Otoño 2026

Hoja de ruta de la clase

Primera parte: las herramientas del día

- 1 **¿Por qué métodos de ensamble?** Conexión con CART (Tutorial 11) y el problema de la *alta varianza* de un árbol individual.
- 2 **Bootstrap como ingrediente:** remuestreo con reposición.
- 3 **Bagging:** promediar B árboles entrenados sobre muestras bootstrap.
- 4 **Random Forest:** bagging + decorrelación de árboles vía sorteo de predictores.
- 5 **Feature importance** e hiperparámetros en `sklearn`.

Segunda parte: paper aplicado

Riascos & Serna (2017, PMLR) – Random forests para predecir días de hospitalización en Colombia.

Tercera parte: notebook en Python

Tut12_Bagging_RF.ipynb con `scikit-learn`: `BaggingRegressor` y `RandomForestRegressor` sobre Boston.

Parte I

Del árbol individual al *bosque*: idea, algoritmo, hiperparámetros

¿Por qué métodos de ensamble?

- En el Tutorial 11 vimos que CART captura no linealidades e interacciones *sin* imponer forma funcional, pero pagamos un costo: el árbol es **inestable**.
- “Inestable” quiere decir: si cambiamos un poquito la muestra de entrenamiento, el árbol estimado *cambia mucho*. En lenguaje del trade-off *sesgo-varianza*: **sesgo bajo, varianza alta**.
- Los *métodos de ensamble* parten de una idea simple de la estadística: **promediar reduce varianza**. Si tengo B predictores independientes con varianza σ^2 , el promedio tiene varianza σ^2/B .
- Estrategia: en lugar de un solo árbol, entrenamos **muchos** sobre versiones perturbadas de la muestra y los promediamos.

Bootstrap: el ingrediente para fabricar muestras

- Dada una muestra de tamaño n , una **muestra bootstrap** es una muestra de tamaño n tomada *con reposición* de la original.
- Consecuencia: cada muestra bootstrap deja afuera $\approx 1/e \approx 37\%$ de las observaciones originales (las *out-of-bag*, ver más adelante).
- Repitiendo el procedimiento B veces, tenemos B muestras *distintas* (pero correlacionadas) que sirven para entrenar B modelos.

Idea clave

Bootstrap simula variabilidad muestral sin tener que recolectar más datos: es el motor de todos los ensambles basados en árboles.

Bagging: *Bootstrap Aggregating* (Breiman, 1996)

La idea: entrenamos B árboles, cada uno sobre una muestra bootstrap distinta, y promediamos sus predicciones.

Algoritmo en tres pasos

- 1 Para $b = 1, \dots, B$: tomar una muestra bootstrap Z^{*b} de la muestra original.
- 2 Sobre Z^{*b} , entrenar un árbol **sin podar** (alta varianza, bajo sesgo): $\hat{f}^{*b}$.
- 3 Combinar:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x) \quad (\text{regresión})$$

o *voto mayoritario* entre las clases predichas (clasificación).

- Cuanto más *distintos* sean los árboles entre sí, más reduce la varianza el promedio.
- ¿Cuántos árboles? B no es un hiperparámetro a calibrar: cuanto más grande, mejor (hasta saturar). Típicamente $B \in [100, 500]$.

¿Por qué bagging no alcanza?

El problema: los árboles están correlacionados

Si hay un predictor **muy fuerte**, *todos* los árboles tienden a partir primero por esa variable. Árboles parecidos $\Rightarrow$ promediar gana *poco*.

- Con correlación promedio ρ entre árboles, la varianza del promedio es:

$$\rho\sigma^2 + \frac{(1-\rho)}{B}\sigma^2.$$

- Cuando $B \rightarrow \infty$ el segundo término se va a cero, pero el primero queda fijo en $\rho\sigma^2$. **Para bajar más la varianza hay que reducir ρ** — esa es la motivación de *random forests*.

Random Forest (Breiman, 2001)

Idea adicional sobre bagging: en cada *split* de cada árbol, no se considera todo el conjunto de predictores; solo un **subconjunto aleatorio** de $m < p$ variables.

¿Por qué funciona?

Forzamos a que los árboles “miren para otro lado” cuando construyen los splits. Las variables más fuertes ya no monopolizan el split raíz $\Rightarrow$ los árboles quedan **más distintos entre sí** (menos correlacionados) $\Rightarrow$ promediar reduce más la varianza.

- Convenciones (Breiman):
 - **Clasificación:** $m \approx \sqrt{p}$.
 - **Regresión:** $m \approx p/3$.
- Casos extremos: $m = p$ recupera *bagging* (no hay sorteo); $m = 1$ da splits casi aleatorios (máxima decorrelación, árboles muy débiles).
- En la práctica, m se elige por **cross-validation** (lo vamos a hacer en el notebook).
- **Random Forest:** *flexibilidad, modelo de caja negra, captura no linealidades.*

Feature importance: interpretabilidad recuperada

Un árbol individual era *interpretable* (se lee como reglas *if-then*). El bosque, en cambio, son cientos de árboles promediados: no se “lee”.

Variable importance scores

Para cada predictor X_j se calcula la **reducción promedio de impureza** (Gini en clasif., suma de cuadrados en regresión) que produce cada vez que aparece en un split, sumada sobre todos los árboles.

- Da un *ranking* de variables: cuáles son las que más usa el bosque.
- Atención: el score es **relativo**, no un efecto causal ni un coeficiente. Mide *poder predictivo*, no *importancia económica*.
- Sesgo conocido: las variables con muchos niveles (continuas, categóricas con muchas categorías) tienden a aparecer con scores más altos. Hay alternativas (*permutation importance*) que corrigen este sesgo.
- Es la herramienta clásica para *interpretar* un random forest (lo vamos a usar en el paper aplicado).

Argumento	Qué controla	Comentario
<code>n_estimators</code>	Cantidad de árboles B	No se calibra: cuanto más, mejor. $B \in [100, 500]$ suele bastar.
<code>max_features</code>	m , predictores sorteados por split	El hiperparámetro de RF. 'sqrt' (clasif.) y 1/3 (regr.) son defaults. Calibrar por CV.
<code>max_samples</code>	Tamaño de cada muestra bootstrap	Default: n (igual al original). Reducirlo acelera y a veces mejora.
<code>max_depth</code>	Profundidad máxima de cada árbol	Default: sin límite. Los árboles del ensemble son <i>grandes y sin podar</i> a propósito.

- En `sklearn.ensemble`: `BaggingRegressor`, `RandomForestRegressor` (y sus versiones `Classifier`). RF **no** requiere estandarizar X .

CART vs. Bagging vs. Random Forest

	CART	Bagging	Random Forest
¿Cuántos árboles?	Uno	B árboles	B árboles
¿Muestra de entrenamiento?	La original	B muestras bootstrap	B muestras bootstrap
¿Predictores en cada split?	Todos los p	Todos los p	$m < p$ sorteados
Sesgo	Bajo	Bajo	Bajo
Varianza	Alta	Media	Baja
Interpretabilidad	Alta (reglas)	Baja (sólo <i>importance</i>)	Baja (sólo <i>importance</i>)

- En **predicción**, RF suele ganarle a bagging, que a su vez le gana a un único CART.
- En **interpretación**, el orden es al revés: CART se lee, los bosques no.
- La pregunta práctica es: *¿necesito predecir o necesito explicar?* Las dos cosas son legítimas en economía aplicada.

Ventajas y desventajas de Random Forest

Ventajas

- **Performance predictiva** muy competitiva (buen desempeño incluso sin ajustar hiperparámetros).
- Captura interacciones y no linealidades, igual que CART.
- No requiere estandarizar.
- Robustísimo a hiperparámetros: $\sqrt{p}$.

Desventajas

- Pierde la interpretabilidad de CART (sólo *importance*).
- Computacionalmente más caro (B árboles).
- Predicción *discontinua* (heredado de CART).
- Sin teoría de inferencia estándar.
- Sesgo *importance* hacia variables con muchos niveles.

- La *otra* familia de ensambles — *boosting* (AdaBoost, Gradient Boosting, XGBoost) — usa una idea distinta: en vez de promediar árboles independientes, los entrena **secuencialmente**, corrigiendo los errores del anterior. Lo vemos en el **Tutorial 13**.

Parte II

Random Forest en economía aplicada

Paper aplicado

Álvaro Riascos & Natalia Serna (2017)

Predicting annual length-of-stay and its impact on health

Proceedings of Machine Learning Research, Medical Informatics and Healthcare, 27–34.

Pregunta y motivación

- Los **días totales de hospitalización por año** (*length-of-stay*, LOS) son un *driver* clave del gasto en salud y de la planificación de capacidad hospitalaria.
- **Pregunta del paper:** ¿se puede *predecir* el LOS anual de un paciente a partir de su historia clínica y demográfica, usando datos administrativos del sistema de salud colombiano?
- **Por qué importa para nosotros:** es una *aplicación predictiva pura* de random forest con datos reales latinoamericanos y comparación contra benchmarks econométricos.
- En el espíritu de Kleinberg, Ludwig, Mullainathan & Obermeyer (2015, AER), este es un *prediction policy problem*: la pregunta es “a quién predecir alto”, no “cuál es el efecto causal de un tratamiento”.

Pregunta empírica

Dado un panel administrativo del sistema de salud colombiano, ¿cuán bien podemos predecir el LOS anual usando random forests, comparado con modelos lineales?

- **Fuente:** registros administrativos de una aseguradora privada de salud en Colombia. Panel de pacientes con historia clínica, demografía y consumo de servicios.
- **Outcome** (Y): días totales de hospitalización en el año (*length-of-stay*). Variable continua con **cola larga** (la mayoría tiene cero o pocos días, pocos pacientes muchos).
- **Predictores** (X): edad, sexo, diagnósticos previos (códigos CIE-10), consumo de medicamentos, hospitalizaciones del año anterior. Mezcla de continuas, binarias y categóricas con *muchos* niveles.
- **Dificultades:**
 - Alta dimensionalidad efectiva (cientos de códigos diagnósticos).
 - Fuertes *interacciones* (la edad importa distinto según el diagnóstico).
 - Outliers crónicos: muy pocos pacientes acumulan muchos días.

- **Estrategia:** comparar tres modelos en su capacidad de predecir LOS *fuera de la muestra*:
 - OLS / regresión lineal.
 - Árbol de regresión (CART).
 - **Random Forest.**
- Split train/test estándar; reportan *root-mean-squared error* (RMSE) en test.
- Hiperparámetros de RF calibrados por cross-validation: `n_estimators`, `max_features`, `min_samples_leaf`.
- Usan **variable importance scores** para identificar qué características predicen LOS alto.

Por qué RF acá

El problema combina alta dimensionalidad, interacciones fuertes y mezcla de tipos de datos. RF maneja todo eso *sin* pedirle al investigador que especifique la forma funcional ni las interacciones a mano.

Hallazgos

- RF reduce el RMSE de test **sustancialmente** respecto a OLS y a un CART único.
- Variables más importantes: *hospitalizaciones previas*, *diagnósticos crónicos* (cáncer, insuficiencia renal), *edad*, *consumo de medicamentos de alto costo*.
- No sorprende quién predice alto: la novedad es *cuánto* mejora la calidad de la predicción frente a un modelo lineal.

Mensaje metodológico

- En problemas *predictivos puros* (cuánto va a costar, cuántos días, qué demanda esperar), RF suele ser un *default razonable*: pocos hiperparámetros que calibrar, manejo natural de heterogeneidad, performance competitiva.
- La *interpretabilidad* se recupera vía *variable importance*, no vía coeficientes.
- Conexión con la agenda de *prediction policy problems*: en gestión hospitalaria, predecir bien es *en sí mismo* una contribución a la política, sin necesidad de hablar de causalidad.

Qué nos llevamos

- **Bagging** reduce la varianza de un árbol promediando B árboles bootstrap; **Random Forest** agrega el sorteo de m predictores por split para decorrelacionar los árboles y reducir aún más la varianza.
- En **Riascos & Serna (2017)** vimos un caso típico: alta dimensionalidad, interacciones fuertes y outliers. RF aporta donde el modelo lineal no llega, sin pedirle al investigador que especifique la forma funcional.
- RF es un *default razonable* en problemas predictivos aplicados: pocos hiperparámetros, manejo natural de heterogeneidad, performance competitiva. La *interpretabilidad* se recupera vía *feature importance*.

Próximo tutorial

En el **Tutorial 13** vamos a ver *boosting* y *XGBoost*: la otra gran familia de ensambles, donde los árboles se entrenan **secuencialmente** corrigiendo los errores del anterior (en vez de promediar árboles independientes).

Ahora vamos a implementar bagging y random forest en Python.

Notebook: Tut12_Bagging_RF.ipynb

Datos:

- Boston (de ISLP): regresión sobre el valor mediano de viviendas en barrios de Boston.

Herramientas que vamos a usar:

- `sklearn.ensemble.BaggingRegressor` y `RandomForestRegressor`.
- `sklearn.tree.DecisionTreeRegressor` como *baseline* (repaso de Tut11).
- `sklearn.model_selection.GridSearchCV` para elegir `n_estimators` y `max_features`.
- `feature_importances_` para el ranking de variables.
- `sklearn.metrics.mean_squared_error` para la comparación final.

- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24, 123–140.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Breiman, L. (2003). Statistical modeling: the two cultures. *Quality Control and Applied Statistics*, 48(1), 81–82.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: with applications in Python*. Cap. 8.2. Springer.
- Friedman, J., Hastie, T., & Tibshirani, R. (2001). *The elements of statistical learning*. Cap. 15. Springer.
- Riascos, A., & Serna, N. (2017). Predicting annual length-of-stay and its impact on health. *Medical Informatics and Healthcare*, PMLR, 27–34.
- Sosa Escudero, W. (2021). *Big Data*, 7.^a edición, Siglo XXI, Cap. 3, pp. 85–94.